COMSATS University Islamabad

Department of Computer Science

CS332 – Parallel & Distributed Computing

LAB 06

Understanding the Network Time Protocol

Student	Abrar Butt
QALAM ID	460297
Instructor	Dr. Shah Khalid
Lab Instructor	Daniyal Attiq
Date	March 02, 2026
Class	BSCS — 13 — C

GitHub Repository:

https://github.com/Abrarbut/parallel-and-computing-distribution.git

Contents

1 Introduction **2**

2 Objectives **2**

3 Tools / Software **2**

4 Description **2**

4.1 Key Features of NTP 3

5 Lab Tasks **4**

5.1 Task 1 – Single Server NTP Client 4

5.2 Task 2 – Ten Server NTP Client 7

6 Program Output **10**

7 Code Files **11**

8 Deliverables **11**

1. Introduction

This lab focuses on giving an overview of the **Network Time Protocol (NTP)** and its implementation. NTP is an application-layer protocol responsible for synchronizing the clocks of hosts on a TCP/IP network. It was developed by **David Mills** in 1981 at the University of Delaware and operates over **UDP port 123**.

2. Objectives

- To understand the concept of the Network Time Protocol.
- To implement an NTP client that queries a single time server.
- To extend the client to query **10 different** NTP servers and display the synchronized time from each.

3. Tools / Software

- Microsoft Visual Studio 2013
- C++ with Winsock2 (`ws2_32.lib`)
- Windows OS with internet access

4. Description

Network Time Protocol (NTP) is a networking protocol for clock synchronization between computer systems over packet-switched, variable-latency data networks. It uses a hierarchical system of time sources called *strata*:

- **Stratum 0** – Highly accurate atomic or GPS clocks.
- **Stratum 1** – Servers connected directly to Stratum 0 hardware.
- **Stratum 2+** – Servers synchronizing from higher strata.

NTP packets are **48 bytes** long. The client sets the `li_vn_mode` field to `0x1B` (LI=0, Version=3, Mode=3/client) and sends a UDP datagram to port 123. The server responds with the transmit timestamp (`tx_ts_sec`) in *NTP epoch* (seconds since 1 January 1900). Subtracting the constant $\Delta = 2{,}208{,}988{,}800$ converts it to *Unix epoch* (seconds since 1 January 1970).

4.1 Key Features of NTP

Table 1: Network Time Protocol (NTP) Specifications

Feature	**Description**
Protocol Layer	Application Layer (Transport: UDP)
Port Number	`123`
Epoch Offset	`2,208,988,800` seconds From: 1 Jan 1900 00:00 UTC To: 1 Jan 1970 00:00 UTC
Packet Structure	48-byte fixed format `li_vn_mode` field = `0x1B`
Client Request	`0x1B` LI=0 (No leap warning) VN=3 (NTP version 3) Mode=3 (Client mode)
Synchronization	Millisecond accuracy Hierarchical stratum levels

5. Lab Tasks

5.1 Task 1 – Single Server NTP Client

Run the provided `ntp_client.cpp` to fetch and display the current UTC time from `pool.ntp.org`.

Code: ntp_client.cpp

```cpp
#include <iostream>
#include <string>
#include <winsock2.h>
#include <ws2tcpip.h>
#include <time.h>

#pragma comment(lib, "ws2_32.lib")

using namespace std;

typedef struct {
    unsigned char  li_vn_mode;
    unsigned char  stratum;
    unsigned char  poll;
    unsigned char  precision;
    unsigned int   root_delay;
    unsigned int   root_dispersion;
    unsigned int   ref_id;
    unsigned int   ref_ts_sec;
    unsigned int   ref_ts_frac;
    unsigned int   orig_ts_sec;
    unsigned int   orig_ts_frac;
    unsigned int   rx_ts_sec;
    unsigned int   rx_ts_frac;
    unsigned int   tx_ts_sec;
    unsigned int   tx_ts_frac;
} NTPPacket;

#define NTP_PORT 123
#define NTP_TIMESTAMP_DELTA 2208988800ULL

bool GetTimeFromNTPServer(const string& serverName)
{
    SOCKET      sock    = INVALID_SOCKET;
    addrinfo*   result  = NULL;
    bool        success = false;

    addrinfo hints;
    ZeroMemory(&hints, sizeof(hints));
    hints.ai_family   = AF_INET;
    hints.ai_socktype = SOCK_DGRAM;
    hints.ai_protocol = IPPROTO_UDP;

    char portStr[8];
    sprintf_s(portStr, "%d", NTP_PORT);

    if (getaddrinfo(serverName.c_str(), portStr, &hints, &result) != 0) {
```

```
        cerr << "[ERROR] Cannot resolve server: " << serverName << endl;
        return false;
    }

    sock = socket(result->ai_family, result->ai_socktype, result->ai_protocol);
    if (sock == INVALID_SOCKET) {
        cerr << "[ERROR] Socket creation failed." << endl;
        freeaddrinfo(result);
        return false;
    }

    DWORD timeout = 3000;
    setsockopt(sock, SOL_SOCKET, SO_RCVTIMEO, (char*)&timeout, sizeof(timeout));

    NTPPacket packet;
    ZeroMemory(&packet, sizeof(packet));
    packet.li_vn_mode = 0x1B;

    if (sendto(sock, (char*)&packet, sizeof(packet), 0,
               result->ai_addr, (int)result->ai_addrlen) == SOCKET_ERROR)
    {
        cerr << "[ERROR] Failed to send to: " << serverName << endl;
        goto cleanup;
    }

    {
        sockaddr_in fromAddr;
        int fromLen = sizeof(fromAddr);
        int received = recvfrom(sock, (char*)&packet, sizeof(packet), 0,
                                (sockaddr*)&fromAddr, &fromLen);
        if (received == SOCKET_ERROR) {
            cerr << "[ERROR] No response from: " << serverName << endl;
            goto cleanup;
        }
    }

    {
        unsigned long long txSec = ntohl(packet.tx_ts_sec);
        if (txSec < NTP_TIMESTAMP_DELTA) {
            cerr << "[ERROR] Invalid timestamp received." << endl;
            goto cleanup;
        }
        time_t unixTime = (time_t)(txSec - NTP_TIMESTAMP_DELTA);

        struct tm tmInfo;
        gmtime_s(&tmInfo, &unixTime);

        char timeBuf[64];
        strftime(timeBuf, sizeof(timeBuf), "%Y-%m-%d  %H:%M:%S UTC", &tmInfo);

        cout << "Server : " << serverName << endl;
        cout << "Time   : " << timeBuf    << endl;
        cout << "-----------------------------------------" << endl;
        success = true;
    }

cleanup:
    closesocket(sock);
```

```
    freeaddrinfo(result);
    return success;
}

int main()
{
    WSADATA wsaData;
    if (WSAStartup(MAKEWORD(2, 2), &wsaData) != 0) {
        cerr << "WSAStartup failed." << endl;
        return 1;
    }

    cout << "========================================" << endl;
    cout << "  Lab 06 - NTP Client                   " << endl;
    cout << "========================================" << endl << endl;

    string server = "pool.ntp.org";
    GetTimeFromNTPServer(server);

    WSACleanup();
    cout << endl << "Press Enter to exit...";
    cin.get();
    return 0;
}
```

Listing 1: ntp_client.cpp – Task 1 (Original)

5.2 Task 2 – Ten Server NTP Client

The code was modified to query **10 different NTP servers**. The key change in `main()` is replacing the single server string with a `string[10]` array and iterating over it with a `for` loop.

Code: ntp_client_task2.cpp

```cpp
#include <iostream>
#include <string>
#include <winsock2.h>
#include <ws2tcpip.h>
#include <time.h>

#pragma comment(lib, "ws2_32.lib")

using namespace std;

typedef struct {
    unsigned char  li_vn_mode;
    unsigned char  stratum;
    unsigned char  poll;
    unsigned char  precision;
    unsigned int   root_delay;
    unsigned int   root_dispersion;
    unsigned int   ref_id;
    unsigned int   ref_ts_sec;
    unsigned int   ref_ts_frac;
    unsigned int   orig_ts_sec;
    unsigned int   orig_ts_frac;
    unsigned int   rx_ts_sec;
    unsigned int   rx_ts_frac;
    unsigned int   tx_ts_sec;
    unsigned int   tx_ts_frac;
} NTPPacket;

#define NTP_PORT 123
#define NTP_TIMESTAMP_DELTA 2208988800ULL

bool GetTimeFromNTPServer(const string& serverName)
{
    SOCKET      sock    = INVALID_SOCKET;
    addrinfo*   result  = NULL;
    bool        success = false;

    addrinfo hints;
    ZeroMemory(&hints, sizeof(hints));
    hints.ai_family   = AF_INET;
    hints.ai_socktype = SOCK_DGRAM;
    hints.ai_protocol = IPPROTO_UDP;

    char portStr[8];
    sprintf_s(portStr, "%d", NTP_PORT);

    if (getaddrinfo(serverName.c_str(), portStr, &hints, &result) != 0) {
        cerr << "[ERROR] Cannot resolve server: " << serverName << endl;
        return false;
    }
```

```

    sock = socket(result->ai_family, result->ai_socktype, result->ai_protocol);
    if (sock == INVALID_SOCKET) {
        cerr << "[ERROR] Socket creation failed." << endl;
        freeaddrinfo(result);
        return false;
    }

    DWORD timeout = 3000;
    setsockopt(sock, SOL_SOCKET, SO_RCVTIMEO, (char*)&timeout, sizeof(timeout));

    NTPPacket packet;
    ZeroMemory(&packet, sizeof(packet));
    packet.li_vn_mode = 0x1B;

    if (sendto(sock, (char*)&packet, sizeof(packet), 0,
               result->ai_addr, (int)result->ai_addrlen) == SOCKET_ERROR)
    {
        cerr << "[ERROR] Failed to send to: " << serverName << endl;
        goto cleanup;
    }

    {
        sockaddr_in fromAddr;
        int fromLen = sizeof(fromAddr);
        int received = recvfrom(sock, (char*)&packet, sizeof(packet), 0,
                                (sockaddr*)&fromAddr, &fromLen);
        if (received == SOCKET_ERROR) {
            cerr << "[ERROR] No response from: " << serverName << endl;
            goto cleanup;
        }
    }

    {
        unsigned long long txSec = ntohl(packet.tx_ts_sec);
        if (txSec < NTP_TIMESTAMP_DELTA) {
            cerr << "[ERROR] Invalid timestamp received." << endl;
            goto cleanup;
        }
        time_t unixTime = (time_t)(txSec - NTP_TIMESTAMP_DELTA);

        struct tm tmInfo;
        gmtime_s(&tmInfo, &unixTime);

        char timeBuf[64];
        strftime(timeBuf, sizeof(timeBuf), "%Y-%m-%d  %H:%M:%S UTC", &tmInfo);

        cout << "Server : " << serverName << endl;
        cout << "Time   : " << timeBuf    << endl;
        cout << "-----------------------------------------" << endl;
        success = true;
    }

cleanup:
    closesocket(sock);
    freeaddrinfo(result);
    return success;
}
```

```cpp

int main()
{
    WSADATA wsaData;
    if (WSAStartup(MAKEWORD(2, 2), &wsaData) != 0) {
        cerr << "WSAStartup failed." << endl;
        return 1;
    }

    cout << "========================================" << endl;
    cout << "  Lab 06 - NTP Client (10 Servers)      " << endl;
    cout << "========================================" << endl << endl;

    // Task 2: Query 10 NTP servers
    string servers[10] = {
        "pool.ntp.org",
        "time.google.com",
        "time.windows.com",
        "time.apple.com",
        "time.cloudflare.com",
        "0.pool.ntp.org",
        "1.pool.ntp.org",
        "2.pool.ntp.org",
        "3.pool.ntp.org",
        "ntp.ubuntu.com"
    };

    int successCount = 0;
    for (int i = 0; i < 10; i++)
    {
        cout << "[" << (i + 1) << "/10] Querying: " << servers[i] << endl;
        if (GetTimeFromNTPServer(servers[i]))
            successCount++;
    }

    cout << endl;
    cout << "========================================" << endl;
    cout << "  Completed: " << successCount << "/10 servers responded." << endl;
    cout << "========================================" << endl;

    WSACleanup();
    cout << endl << "Press Enter to exit...";
    cin.get();
    return 0;
}
```

Listing 2: ntp_client_task2.cpp – Task 2 (10 Servers)

6. Program Output

The following output was captured by running `ntp_client_task2.cpp` in Microsoft Visual Studio 2013 on 2 March 2026. **8 out of 10 servers responded** successfully. `time.apple.com` and `ntp.ubuntu.com` timed out (3-second timeout per server).

Output (Run 1 – 2026-03-02 19:29 UTC):

```
==========================================
  Lab 06 - NTP Client (10 Servers)
==========================================

[1/10] Querying: pool.ntp.org
Server : pool.ntp.org
Time   : 2026-03-02  19:29:19 UTC
-------------------------------------------
[2/10] Querying: time.google.com
Server : time.google.com
Time   : 2026-03-02  19:29:19 UTC
-------------------------------------------
[3/10] Querying: time.windows.com
Server : time.windows.com
Time   : 2026-03-02  19:29:19 UTC
-------------------------------------------
[4/10] Querying: time.apple.com
[ERROR] No response from: time.apple.com
[5/10] Querying: time.cloudflare.com
Server : time.cloudflare.com
Time   : 2026-03-02  19:29:23 UTC
-------------------------------------------
[6/10] Querying: 0.pool.ntp.org
Server : 0.pool.ntp.org
Time   : 2026-03-02  19:29:23 UTC
-------------------------------------------
[7/10] Querying: 1.pool.ntp.org
Server : 1.pool.ntp.org
Time   : 2026-03-02  19:29:23 UTC
-------------------------------------------
[8/10] Querying: 2.pool.ntp.org
Server : 2.pool.ntp.org
Time   : 2026-03-02  19:29:23 UTC
-------------------------------------------
[9/10] Querying: 3.pool.ntp.org
Server : 3.pool.ntp.org
Time   : 2026-03-02  19:29:23 UTC
-------------------------------------------
[10/10] Querying: ntp.ubuntu.com
[ERROR] No response from: ntp.ubuntu.com

==========================================
  Completed: 8/10 servers responded.
==========================================

Press Enter to exit...
```

7. Code Files

- `ntp_client.cpp` – Task 1, original single-server client
- `ntp_client_task2.cpp` – Task 2, modified 10-server client
- `lab06_output.txt` – Console output from execution

GitHub Repository:
https://github.com/Abrarbut/parallel-and-computing-distribution.git

8. Deliverables

Submit the complete project folder (containing both `.cpp` files and this lab report) on LMS. Each group should upload one submission.